TOPDEVCREW · PRUEBA TÉCNICA

Asistente RAG documental con ingesta asíncrona y trazabilidad de extremo a extremo

Ingeniero AI Full Stack Semi-Senior · Entrégala antes del 6 de noviembre de 2026 a las 16:48 UTC

Cómo funciona

- Tienes 72 horas desde que te enviamos este proyecto.
- Puedes usar IA generativa (Claude, Copilot, ChatGPT…) como quieras.
- Solo puedes entregar UNA VEZ. Al pegar el link, congelamos el último commit de tu repositorio en ese momento. Esa es la versión que evaluamos y la que revisamos en la entrevista; los commits posteriores se ignoran y no hay una segunda entrega. Asegúrate de que el repositorio esté listo antes de pegar el link.
- Sube tu trabajo a UN repositorio PÚBLICO de GitHub. Si el proyecto tiene varias partes (backend, frontend, infraestructura), van juntas en ese único repositorio (monorepo). No aceptamos repositorios privados, varios repositorios ni archivos zip.
- En la entrevista técnica vas a recorrer tu código en tu propia pantalla y defender tus decisiones. Entrega solo lo que entiendas.
- Donde algo sea ambiguo, decide y explica por qué en DECISIONS.md.

El proyecto

Portalis Logística, una empresa ficticia de transporte y almacenaje, tiene cientos de PDFs y documentos DOCX con manuales operativos, contratos con clientes y procedimientos de aduana, dispersos en carpetas compartidas. Los operadores hacen preguntas por chat y hoy reciben respuestas sin fuente. Construye un MVP de asistente interno: se suben documentos, se procesan en segundo plano, se indexan y se consultan en lenguaje natural con respuestas que citan fragmentos y páginas concretas. El sistema debe poder cambiar de proveedor de LLM sin tocar la lógica de negocio y debe permitir diagnosticar por qué una respuesta fue lenta o mala. Entregas un monorepo con backend, frontend e infraestructura local con docker compose.

REQUISITOS

- API en FastAPI con modelos Pydantic estrictos para subir documentos, consultar el estado de ingesta, listar documentos y hacer preguntas.
- Pipeline de ingesta con Celery y Redis: parsing con Docling, troceado con metadatos (documento, página, sección), generación de embeddings y almacenamiento en PostgreSQL con pgvector. La tarea debe ser idempotente y reintentable.
- Endpoint de consulta RAG que recupere fragmentos relevantes, genere la respuesta con LiteLLM (Gemini como proveedor principal y un segundo proveedor o un proveedor simulado como respaldo) y devuelva la respuesta con citas verificables.
- Salida estructurada del paso de generación (respuesta, citas, nivel de confianza, indicador de que no hay información suficiente) validada con Pydantic AI o, si lo descartas, con una alternativa justificada.
- Comportamiento definido cuando la recuperación no encuentra evidencia suficiente: el sistema debe decidir si responde, se abstiene o pide aclaración, y hacerlo de forma consistente.
- Frontend en React con subida de archivos, estado de ingesta en vivo (polling o streaming), chat con las citas visibles y apertura del fragmento fuente.

- Flower u otra herramienta equivalente en el docker compose para inspeccionar colas y tareas, con una breve nota de cómo usarla para diagnosticar una ingesta atascada.
- Modo de ejecución sin claves reales (proveedor de LLM y embeddings simulados y deterministas) para que cualquiera pueda levantar el proyecto y correr los tests sin credenciales.

NIVEL DE CALIDAD

- Tests automatizados: unitarios para troceado y construcción del prompt, integración contra PostgreSQL con pgvector real en contenedor, y pruebas del flujo de ingesta con Celery. El LLM se simula en CI y se incluye un pequeño conjunto de preguntas de evaluación con documentos de ejemplo para medir la calidad de recuperación.
- Manejo de errores: reintentos con backoff, tratamiento de documentos corruptos o protegidos, estados de ingesta claros (pendiente, procesando, fallido, listo), timeouts y fallback entre proveedores de LLM, y errores de API consistentes.
- Observabilidad con OpenTelemetry: una traza que atraviese API, tarea de Celery, recuperación en pgvector y llamada al LLM, con atributos útiles (modelo, tokens, latencia, número de fragmentos) y sin registrar contenido sensible de los documentos.
- Seguridad: validación de tipo y tamaño de archivo, secretos solo por variables de entorno, tratamiento del contenido de los documentos como no confiable frente a prompt injection, y aislamiento básico por espacio de trabajo o tenant en las consultas.
- Reproducibilidad: un solo comando levanta el stack completo y un workflow de GitHub Actions ejecuta lint y tests en cada push.

DECIDE Y JUSTIFICA

- ¿Qué estrategia de troceado y de recuperación eliges (tamaño, solapamiento, búsqueda híbrida, reranking) y cómo justificas con tu conjunto de evaluación que es la adecuada para este tipo de documentos?
- Si dos documentos o versiones se contradicen, o una respuesta tiene poca evidencia, ¿cómo lo comunica el sistema al operador y qué cambias en la UX?
- ¿Cuál es tu política de proveedor y coste: cuándo haces fallback, cómo limitas tokens y latencia y qué compromisos aceptas entre calidad y precio con Gemini?
- ¿Qué datos de los documentos pueden aparecer en trazas, logs y métricas, y cómo equilibras diagnóstico y confidencialidad?

ENTREGABLES

- Un único repositorio público de GitHub con todo el proyecto en formato monorepo (backend, frontend e infraestructura con docker compose). Se evalúa únicamente el commit más reciente del repositorio en el momento de tu entrega, así que asegúrate de que ese estado esté completo y funcione.
- README con instrucciones para ejecutarlo (incluido el modo sin claves), qué está hecho, qué no y limitaciones conocidas.
- DECISIONS.md con tu postura sobre las preguntas abiertas, los trade-offs, las alternativas descartadas y cómo usaste la IA generativa (qué aceptaste, qué corregiste).
- Pruebas automatizadas y el conjunto pequeño de evaluación de recuperación con instrucciones para correrlo.
- Workflow de GitHub Actions que ejecute lint y tests, y una breve captura o descripción de una traza de OpenTelemetry de punta a punta.

Entrega

Pega el link de tu repositorio público de GitHub en:

https://app.topdevcrew.com/test/d3209cddb7d129af912e09463e200c29